MODULE 14 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Watermarks, Late Data and Triggers

How long a window waits, what happens to stragglers, and when results are emitted.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 14 of 45

admin@sthithapragna.com

A watermark is the pipeline's best guess that **everything for this window has arrived**.

Where the watermark comes from

- Sources estimate it. For Pub/Sub, from the **oldest unacknowledged message**.
- By default Pub/Sub watermarks use **publish time**, not when the reading was taken.
- Set a **timestamp attribute**, and the watermark follows **event time** instead.
- An **idle source** holds back the watermark for the whole pipeline.

> FOR LARKSPUR
>
> If the head-end publishes in batches, publish time and reading time can be hours apart.

On time or late?

ON TIME

Before the watermark passes

- Arrives before the window closes
- Included in the main result
- The normal case
- 96% of Larkspur's readings

LATE

After the watermark passes

- Event time is inside a closed window
- Dropped, by default
- Needs allowed lateness to count
- 4% of Larkspur's, after an hour

THE TRAP

By default, late data is silently dropped. Totals look plausible and are quietly wrong.

Keeping the door open

- **Allowed lateness** keeps a window open for a set time after the watermark passes.
- Late elements within it **update the result**, through late firings.
- After the watermark passes the window end **plus** the lateness, the window expires.
- Every open window **holds state**. Longer lateness costs memory and money.

> THE TIEBREAKER
>
> Match allowed lateness to how late data really arrives, and to how much the late part matters.

Four kinds of trigger

TRIGGER	FIRES WHEN	EXAMPLE
Event time	The watermark passes a point	AfterWatermark: the window closes
Processing time	Wall-clock time passes	AfterProcessingTime: every minute
Data-driven	Enough elements arrive	AfterCount: every 1,000 readings
Composite	Rules combine other triggers	Repeatedly, AfterEach, AfterAny, AfterAll

REMEMBER

The default is a single event-time firing when the watermark passes the end of the window.

One window, several results

01

Early panes

Provisional totals every minute, while the half hour runs.

02

On-time pane

When the watermark passes the end of the window.

03

Late panes

Each time a straggler arrives, within the allowed lateness.

04

Expiry

Window end plus lateness: state is dropped.

05

Too late

Anything later is dropped, or caught by a batch fix.

HOW IT WORKS

Each result is called a pane. Downstream systems must expect more than one per window.

Accumulating or discarding panes?

ACCUMULATING

Each pane is the full total so far

- Late pane: the new, complete total
- Downstream overwrites the row
- Right for dashboards and upserts
- Simple to read

DISCARDING

Each pane is only what is new

- Late pane: just the extra readings
- Downstream must add panes up
- Right for append-and-sum sinks
- Overwriting would lose data

THE TIEBREAKER

Overwriting sink: accumulate. Adding sink: discard. Mixing them up corrupts the totals.

Westgrid's half-hour demand

```
1 demand = (by_region
2     | "Half hours" >> beam.WindowInto(
3         window.FixedWindows(30 * 60),
4         trigger=trigger.AfterWatermark(
5             early=trigger.AfterProcessingTime(60),
6             late=trigger.AfterCount(1)),
7         allowed_lateness=72 * 60 * 60,
8         accumulation_mode=trigger.AccumulationMode.ACCUMULATING)
9     | "Sum" >> beam.CombinePerKey(sum))
```

4-6 A provisional total each minute, a final at close, an update per straggler.

7-8 Wait 72 hours for late meters. Each pane is the full total.

HOW IT WORKS

The sink upserts by region and half hour, so each accumulating pane replaces the last.

How late is too late?

- Regions are few, so 72 hours of state for **region totals** is cheap.
- Per-meter windows mean **3.1 million keys** held open for 72 hours. Expensive.
- Option: short lateness in streaming, then a **nightly batch recompute** in BigQuery.
- Fast and nearly right now, exactly right by morning.

> THE TIEBREAKER
>
> Only 0.3% of readings are a day late. Paying streaming state for them may not be worth it.

Watermark trouble

- **Data freshness**: how far behind event time the pipeline is. Rising means a lagging watermark.
- **System latency**: how long an item has waited or been processing.
- Common causes: an **idle source**, a **stuck step**, or a sink that will not accept writes.
- Results never appear? Check the watermark before adding workers.

> THE SIGNAL
>
> Windows wait for the watermark. A stuck watermark looks like a pipeline that produces nothing.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Late readings **missing** from totals	Allowed lateness, with late firings
Provisional results before the window closes	Early firings on processing time
Sink **overwrites** each window's row	Accumulating panes
Sink **adds up** every pane	Discarding panes
Windows use **publish time**, not reading time	A timestamp attribute on messages
Data freshness keeps rising	A lagging watermark: idle source or stuck step

THE PATTERN

Watermark says when. Lateness says how long. Trigger says how often. Accumulation says what each result means.

> Larkspur's half-hour region totals use fixed windows on event time, with the default trigger. Readings that arrive two or three hours late never appear in any total.
>
> The totals must eventually include every reading that arrives within three days.

What should the team change?

A Switch to processing-time windows, so late readings land somewhere

B Add more workers, so that the pipeline keeps up with the backlog

C Allow 72 hours of lateness, with a late firing for each straggler

D Use session windows per region, so that windows stay open longer

◆ PAUSE AND DECIDE

What should the team change?

A Processing-time windows

Late readings would land, but in the wrong half hour.

B More workers

The readings are late at the meter. Workers cannot make them earlier.

C 72 hours of lateness

Windows stay open for the stragglers, and each one updates the total.

D Session windows

Regions never go quiet, so sessions would never close.

THE SIGNAL

'Late readings never appear' with the default trigger means zero allowed lateness. Add lateness and late firings.

> After adding late firings, Westgrid's dashboard sometimes shows a half-hour total collapse to a tiny number when a straggler arrives.
>
> The sink overwrites the row for each region and half hour with whatever the latest pane says. The pipeline uses discarding mode.

What should the team change?

A Turn off late firings, so that only the on-time pane is written

B Increase the allowed lateness, so stragglers arrive inside it

C Make the sink append every pane as a new row in the table

D Switch the pipeline's accumulation mode to accumulating

◆ PAUSE AND DECIDE

What should the team change?

A **No late firings**

The dashboard stops collapsing, but late readings vanish again.

B **More lateness**

The stragglers are already inside it. That is why panes fire.

C **Append every pane**

Correct only if every reader sums the rows. The dashboard reads one.

D **Accumulating mode**

Each late pane carries the full new total, so overwriting is safe.

THE SIGNAL

An overwriting sink plus discarding panes loses data. Overwrite means accumulate.

> Larkspur's control room wants a provisional demand figure for the current half hour every minute, while it is still running.
>
> Westgrid still needs one settled figure as soon as each half hour closes.

How should the trigger be set?

A Sliding windows of 30 minutes, starting every minute

B Fixed 1-minute windows, summed by the control room's dashboard

C Early firings every minute, inside the watermark trigger

D A data-driven trigger that fires after every 1,000 readings

◆ PAUSE AND DECIDE

How should the trigger be set?

A Sliding windows

Rolling half hours, not the current settlement period.

B 1-minute windows

Pushes the summing onto the dashboard, and loses the settled figure.

C Early firings

Provisional totals every minute, then the on-time figure at close.

D Every 1,000 readings

Fires by volume, so quiet minutes give no update at all.

THE SIGNAL

'Provisional, every minute' plus 'settled at close' means early firings inside AfterWatermark.

> Larkspur's head-end collects readings and publishes them to Pub/Sub in batches, sometimes hours after the meters took them. Each message body holds reading_ts.
>
> The half-hour windows behave as if readings happened when the head-end published them.

What fixes the windows?

A. Increase the allowed lateness, so that the batches still count

B. Use processing-time triggers, so that the windows close on time

C. Publish with ordering keys, so the batches arrive in order

D. Put reading_ts in an attribute, and read it as the timestamp

◆ PAUSE AND DECIDE

What fixes the windows?

A More lateness

The readings are not late. They are being stamped with the wrong time.

B Processing-time triggers

Changes when results fire, not which window a reading joins.

C Ordering keys

Order does not change the timestamp each message carries.

D A timestamp attribute

Messages carry event time, so windows and the watermark follow it.

THE SIGNAL

Windows following publish time means the source has no event timestamp. Give it one.

A per-meter streaming pipeline keeps 3.1 million meters' windows open for 72 hours of allowed lateness. Its state, and its bill, keep growing.

Only 0.3% of readings arrive more than a day late. Billing is recalculated from BigQuery every night anyway.

What is the best change?

A Shorten the lateness, letting the nightly batch catch the rest

B Keep 72 hours of lateness, and move to larger worker machines

C Set the allowed lateness to zero, and ignore all late readings

D Move every meter into the global window, with no triggers at all

◆ PAUSE AND DECIDE

What is the best change?

A Short lateness, nightly fix

Streaming covers most late data cheaply; the batch catches the rest.

B Larger machines

Pays more to hold state for a tiny fraction of readings.

C Zero lateness

Drops the 4% that arrive after an hour, every day.

D Global, no triggers

A stream in the global window with no trigger never emits.

THE SIGNAL

Expensive state for a tiny late tail, plus an existing nightly recompute: shorten the lateness.

A Larkspur pipeline reads two Pub/Sub subscriptions: live readings, and a test feed that has been silent for a week.

Its data freshness rises steadily, and its windowed results have stopped appearing, though readings keep arriving.

What should the team do?

A Add more workers, so the pipeline can catch up with the readings

B Remove the silent test feed, or move it into a separate pipeline

C Raise the allowed lateness, so windows wait longer for the data

D Switch the job to at-least-once mode, to reduce its latency

◆ PAUSE AND DECIDE

What should the team do?

A More workers

Workers are not the bottleneck. The watermark is.

B Remove the idle feed

The idle source stops holding the watermark back, and windows close.

C More lateness

Windows would wait even longer for a watermark that is not moving.

D At-least-once mode

Changes duplicates and cost, not a watermark stuck by an idle source.

THE SIGNAL

Rising freshness, no results, and an idle source: the idle source is holding the watermark.

Module 14 in five lines

- The **watermark** guesses when a window is complete. Late data is dropped by default.
- **Allowed lateness** keeps windows open for stragglers, at the cost of state.
- Triggers: **early** panes on processing time, **on-time** at the watermark, **late** per straggler.
- Overwriting sink: **accumulating**. Adding sink: **discarding**.
- Stamp messages with **event time**. An idle source can freeze the watermark.

COMING UP

Next: module 15, Spark and Hadoop on Google Cloud.